PROOFPASS | NEW FEATURES BLUEPRINT

Live Stage Tamper Demo & Dynamic GitHub README Badge

Target: Hackathon IM-03 (Skill Passport: Verified Skills from Real Work)

1. Executive Summary: Why These Two Features?

ProofPass solves a huge real-world problem: technical resume fraud and unearned GitHub repositories. However, to win top honors at Hackathon IM-03, technical execution must be paired with undeniable proof and real-world portability. We are adding two high-impact features for the final submission:

> 1. Live Tamper Demonstration on Stage: Judges often doubt whether 'cryptographic seals' are real or just a static CSS badge. By intentionally tampering with a skill on stage and showing the system reject it in real-time with an alarm, we prove the authenticity of our SHA-256 state engine beyond question.

> 2. Dynamic GitHub README Badge & Public Link: A skill passport must not remain locked inside our localhost website. By offering dynamic SVG badges for GitHub profiles and public verification links, candidates can showcase verified proof to any recruiter worldwide.

The Hackathon Winning Formula

Judges judge on: Impact (Does it solve resume fraud?), Tech Rigor (Is it cryptographic or just mock?), and Demo Quality (Does the live dem

2. Feature 1: Live Tamper Demonstration on Stage

2.1 Simple Layman Explanation (For Team Members)

Think of a physical passport with an embedded holographic watermark. If someone uses white-out to change their name or stamps a fake visa, border control's ultraviolet scanner immediately lights up RED.

In ProofPass, every passport is mathematically sealed with a SHA-256 cryptographic hash calculated over all verified skills, commit evidence, and state ratings. On stage, we provide a 'Tamper Simulator' button. When clicked, we artificially change a candidate's skill (e.g., Docker from 'Claimed' to 'Verified'). The engine recalculates the hash, detects the mismatch, flashes a crimson alert, and revokes the seal in real time.

2.2 Tech Stack Required

> Frontend Crypto: Native Web Crypto API (window.crypto.subtle.digest) - Built into modern browsers, 0 dependencies.

> Backend Crypto: Python hashlib (hashlib.sha256) inside FastAPI for server-side integrity checking.

> UI & Animations: Lucide React icons (ShieldCheck, ShieldAlert), CSS keyframe pulse/shake animations.

2.3 Mathematical Logic & How It Works

The cryptographic seal is calculated over a deterministic, sorted array of all passport claims. In SHA-256, changing even a single letter completely alters the 64-character hexadecimal hash (the Avalanche Effect):

```
// 1. Data stringification (Canonical Order)
Payload = [applicant_id, issued_at, sorted_skills_with_evidence_hashes]
SealHash = SHA256(JSON.stringify(Payload))

// 2. Integrity Verification (Run on every view / load)
CurrentHash = SHA256(JSON.stringify(Current_Passport_Data))
IsValid = (CurrentHash === Stored_SealHash) // False if anything changed!
```

2.4 Implementation in Existing Codebase

> File to Modify: frontend/src/pages/PassportView.jsx (and engine/stateEngine.js)

> Step 1 (State): Add const [isTampered, setIsTampered] = useState(false); in PassportView.jsx.

> Step 2 (Demo Switch): Add a 'Simulate Tamper' toggle switch visible in demo mode.

> Step 3 (Tamper Handler): When clicked, alter one skill state (e.g. 'Docker' state = 'V') in local state.

> Step 4 (Integrity Engine): Recompute hash using crypto.subtle. Compare with stored sealHash.

> Step 5 (Visual Alert): If mismatch: switch seal badge to Red, show 'TAMPER DETECTED', display mismatch diff.

```
// Frontend implementation snippet (PassportView.jsx)
async function checkIntegrity(passport) {
  const payload = JSON.stringify(passport.skills.map(s => [s.name, s.state, s.evidenceHash]));
  const msgBuffer = new TextEncoder().encode(payload);
  const hashBuffer = await crypto.subtle.digest('SHA-256', msgBuffer);
  const currentHash = Array.from(new Uint8Array(hashBuffer))
    .map(b => b.toString(16).padStart(2, '0')).join('');
  return currentHash === passport.sealHash;
}
```

The 30-Second Stage Demo Script

Speaker: 'Judges often ask: What prevents a student from editing their passport scores in Chrome Inspect Element?'
Action: Toggle 'Simulate Tamper' on screen (Docker changes from Claimed to Verified).
Result: Entire UI flashes crimson. Warning: 'CRITICAL ALERT: CRYPTOGRAPHIC HASH MISMATCH. Passport Revoked.'
Speaker: 'Every passport is sealed with SHA-256. If a single commit hash is altered, the seal shatters.'

3. Feature 2: Shareable Public Link & Dynamic GitHub Badge

3.1 Simple Layman Explanation

GitHub profile READMEs do not allow JavaScript or interactive widgets; they only permit image tags. This feature creates a special API endpoint that generates a live SVG image on the fly. When an applicant pastes this badge into their GitHub README, visitors see their verified skills in real time. Clicking the badge opens their official ProofPass verification page.

3.2 Tech Stack Required

> Backend: FastAPI (Python 3.10+) route returning Response(content=svg, media_type='image/svg+xml').

> Database: SQLite (proofpass.db) query to retrieve passport skills count and match percentage.

> Frontend Modal: ShareModal.jsx with tabs for GitHub Markdown, HTML Embed, and Direct URL.

3.3 How the GitHub Dynamic Badge Works

1. Candidate pastes this markdown in their GitHub profile README:

[![ProofPass Verified](http://localhost:8000/api/v1/badges/passport_123.svg)](http://localhost:5173/passport/passport_123)

2. GitHub requests the SVG image from FastAPI endpoint GET /api/v1/badges/{id}.svg.
3. FastAPI queries SQLite for passport_123, counts Verified skills (e.g., 4 Verified | 92% Match).
4. FastAPI returns an SVG graphic colored in ProofPass burnt orange (#b54610) with HTTP Cache-Control: no-cache.

3.4 Backend Code Implementation (FastAPI)

```
# backend/api/badges.py (or inside main.py)
from fastapi import APIRouter, Response

@app.get('/api/v1/badges/{passport_id}.svg')
def get_badge(passport_id: str):
    passport = get_passport_from_db(passport_id) # Query SQLite
    v_count = sum(1 for s in passport['skills'] if s['state'] == 'V')

    svg = f'''<svg xmlns='http://www.w3.org/2000/svg' width='220' height='28'>
      <rect rx='4' width='100%' height='100%' fill='#1a1918'/>
      <rect rx='4' x='110' width='110' height='100%' fill='#b54610'/>
      <text x='10' y='18' fill='#fff' font-family='sans-serif' font-size='11' font-weight='bold'>ProofPass</te
      <text x='118' y='18' fill='#fff' font-family='sans-serif' font-size='11'>{v_count} Verified Skills</text
    </svg>'''
    return Response(content=svg, media_type='image/svg+xml',
                    headers={'Cache-Control': 'no-cache, no-store, must-revalidate'})
```

3.5 Frontend Code Implementation (Share Modal)

```
// Frontend snippet for ShareModal.jsx
const markdownBadge = `[![ProofPass](${API_BASE}/api/v1/badges/${id}.svg)](${WEB_BASE}/passport/${id})`;

return (
  <div className='share-modal'>
    <h3>Share Your Verified Skill Passport</h3>
    <label>GitHub Profile README Markdown</label>
    <input readOnly value={markdownBadge} />
    <button onClick={() => navigator.clipboard.writeText(markdownBadge)}>Copy Markdown</button>
  </div>
);
```

The 20-Second Badge Stage Demo Script

Speaker: 'A skill passport shouldn't stay trapped on our website. With one click, an applicant generates a live dynamic GitHub badge. As so
Action: Show real GitHub profile with glowing burnt-orange badge. Click it -> Opens live ProofPass verification.

4. Team Responsibility Matrix (Division of Work)

To build these features quickly before the hackathon demo, divide the tasks clearly between Frontend and Backend:

FE-1 **Tamper Simulation UI & State** *[Frontend]*

Add toggle switch and isTampered state in PassportView.jsx. Alter skill and trigger check.

FE-2 **Web Crypto Integrity Function** *[Frontend]*

Write checkIntegrity() using native crypto.subtle. Compare calculated hash with stored sealHash.

FE-3 **Share & Embed Modal** *[Frontend]*

Create ShareModal.jsx with copyable tabs for GitHub Markdown, HTML Embed, and Direct Link.

BE-1 **FastAPI Dynamic Badge Route** *[Backend]*

Add GET /api/v1/badges/{passport_id}.svg endpoint returning SVG with no-cache headers.

BE-2 **Backend Verification Route** *[Backend]*

Implement GET /api/v1/passports/{id}/verify returning JSON { valid: bool, originalHash, currentHash }.

BE-3 **Database Seed Personas** *[Backend]*

Ensure SQLite seed data has 2 demo accounts: 'Alex' (4 Verified) and 'Chad' (Flagged / Contradicted).

5. Hackathon Pitch Cheat Sheet (Judge Q&A)

> Q1: Why not store passports on a blockchain?: Answer: Blockchains introduce gas fees, latency, and unnecessary wallet friction for college students. SHA-256 cryptographic hashes give mathematical unforgeability instantly with zero cost and 5ms latency.

> Q2: Can candidates buy fake GitHub stars or commits to trick your engine?: Answer: Stars are ignored. ProofPass evaluates git blame line authorship percentage, commit intervals, and file hashes. Cloned or forked tutorial repositories hit a strict trust ceiling of 0 (Contradicted).

> Q3: What if someone shares an expired or revoked passport link?: Answer: The public URL checks database validity on every render. If revoked, the page shows an explicit red warning banner stating when and why the passport was revoked.